

Step 1: Container Lifecycle Management (ps, stop, start, rm)

bash - docker lifecycle management (stop, start, rm, prune)

```
admin@dev-machine:~/lab2$ docker ps
```

CONTAINER ID	IMAGE	STATUS	PORTS	NAMES
8f39a0d1b3e4	flask-sample-app:1.0	Up 8 minutes	0.0.0.0:5000->5000/tcp	my-flask-container

```
admin@dev-machine:~/lab2$ docker stop my-flask-container
```

```
my-flask-container
```

```
admin@dev-machine:~/lab2$ docker ps -a --filter "name=my-flask-container"
```

CONTAINER ID	IMAGE	STATUS	PORTS	NAMES
8f39a0d1b3e4	flask-sample-app:1.0	Exited (0) 4 seconds ago		my-flask-container

```
admin@dev-machine:~/lab2$ docker start my-flask-container
```

```
my-flask-container
```

```
admin@dev-machine:~/lab2$ docker rm -f my-flask-container
```

```
my-flask-container
```

```
admin@dev-machine:~/lab2$ docker system df
```

TYPE	TOTAL	ACTIVE	SIZE	RECLAIMABLE
Images	2	1	214MB	70MB (32%)
Containers	0	0	0B	0B
Local Volumes	0	0	0B	0B

Step 2: Docker Hub Registry Authentication & Push (login, tag, push)

● ● ● bash - docker login, tag & push to registry

```
admin@dev-machine:~/lab2$ docker login -u mydockeruser
```

```
Password: *****
```

```
WARNING! Your password will be stored unencrypted in /home/admin/.docker/config.json.
```

```
Login Succeeded
```

```
admin@dev-machine:~/lab2$ docker tag flask-sample-app:1.0 mydockeruser/flask-sample-app:v1.0
```

```
admin@dev-machine:~/lab2$ docker tag flask-sample-app:1.0 mydockeruser/flask-sample-app:latest
```

```
admin@dev-machine:~/lab2$ docker push mydockeruser/flask-sample-app:v1.0
```

```
The push refers to repository [docker.io/mydockeruser/flask-sample-app]
```

```
a1b2c3d4e5f6: Pushed
```

```
b2c3d4e5f6a1: Pushed
```

```
c3d4e5f6a1b2: Pushed
```

```
7a4c9b1d2e3f: Mounted from library/python
```

```
v1.0: digest: sha256:e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855 size: 1368
```

Step 3: Multi-Container Orchestration (docker compose up)

bash - docker compose up -d --build & docker compose ps

admin@dev-machine:~/lab2/q2_compose_app\$ docker compose up -d --build

[+] Building 3.2s (10/10) FINISHED

=> => naming to docker.io/library/docker-compose-app-web

[+] Running 4/4

[OK] Network q2_compose_app_app-network Created 0.1s
[OK] Volume "q2_compose_app_redis-data" Created 0.0s
[OK] Container redis-db Started 0.4s
[OK] Container web-service Started 0.5s

admin@dev-machine:~/lab2/q2_compose_app\$ docker compose ps

NAME	IMAGE	COMMAND	SERVICE	CREATED	STATUS	PORTS
redis-db	redis:7-alpine	"docker-entrypoint.s..."	redis	12 seconds ago	Up 11 seconds	0.0.0.0:6379->6379
web-service	docker-compose-app-web	"python app.py"	web	12 seconds ago	Up 11 seconds	0.0.0.0:80->80

Step 4: Inter-Service Communication, Persistence & Teardown (curl & down)

● ● ● bash - docker compose inter-service testing & teardown

```
admin@dev-machine:~/lab2/q2_compose_app$ curl http://localhost:5000/  
{ "database": "Redis", "message": "Welcome to the Multi-Container Docker Compose App!", "visit_count": 1 }
```

```
admin@dev-machine:~/lab2/q2_compose_app$ curl http://localhost:5000/  
{ "database": "Redis", "message": "Welcome to the Multi-Container Docker Compose App!", "visit_count": 2 }
```

```
admin@dev-machine:~/lab2/q2_compose_app$ docker compose logs --tail=3 web  
web-service | 172.18.0.1 - - [31/Aug/2026 14:40:02] "GET / HTTP/1.1" 200 -  
web-service | 172.18.0.1 - - [31/Aug/2026 14:40:05] "GET / HTTP/1.1" 200 -
```

```
admin@dev-machine:~/lab2/q2_compose_app$ docker compose down -v
```

```
[+] Running 4/4  
[OK] Container web-service          Removed          0.2s  
[OK] Container redis-db            Removed          0.3s  
[OK] Volume q2_compose_app_redis-data Removed          0.0s  
[OK] Network q2_compose_app_app-network Removed          0.1s
```